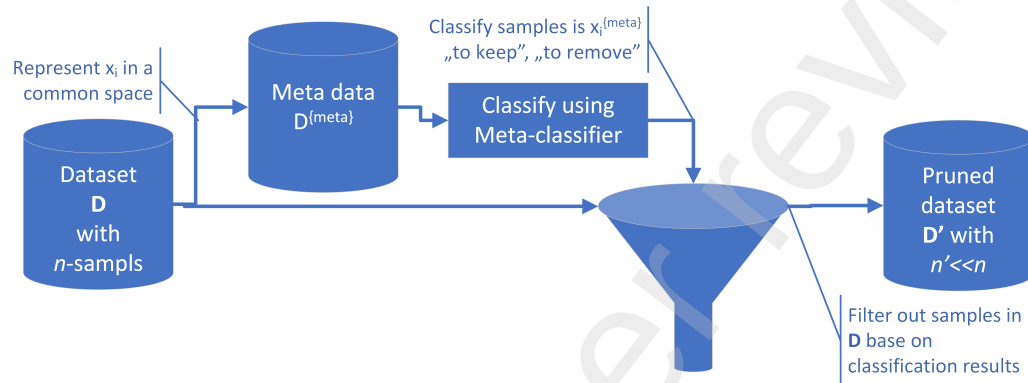


Graphical Abstract

Meta-Instance Selection. Instance Selection as a Classification Problem with Meta-Features.

Marcin Blachnik , Piotr Ciepliński



Highlights

Meta-Instance Selection. Instance Selection as a Classification Problem with Meta-Features.

Marcin Blachnik , Piotr Ciepliński

- A new approach for instance selection is provided that is based on transforming instance selection into a classification problem in meta-feature space
- Meta-feature space can be obtained from the nearest neighbor graph
- Meta-instance selection has a low computational complexity and allows speed-up between 6 to 200 times depending on the reference instance selection method
- Performance in terms of size-reduction ratio of meta-instance selection is comparable to the reference instance selection

Meta-Instance Selection. Instance Selection as a Classification Problem with Meta-Features.

Marcin Blachnik ^{a,*}, Piotr Ciepliński^a

^a*Department of Industrial Informatics, Silesian University of Technology, Akademicka 2A, Gliwice, 44-100, Poland*

Abstract

Data pruning, or instance selection, is an important problem in machine learning especially in terms of nearest neighbour classifier. However, in data pruning which speeds up the prediction phase, there is an issue related to the speed and efficiency of the process itself. In response, the study proposes an approach involving transforming the instance selection process into a classification task conducted in a unified meta-feature space where each instance can be classified and assigned to either the "to keep" or "to remove" class. This approach requires training an appropriate meta-classifier, which can be developed based on historical instance selection results from other datasets using reference instance selection methods as a labeling tool. This work proposes constructing the meta-feature space based on properties extracted from the nearest neighbor graph. Experiments conducted on 17 datasets of varying sizes and five reference instance selection methods (ENN, Drop3, ICF, HMN-EI, and CCIS) demonstrate that the proposed solution achieves results comparable to reference instance selection methods while significantly reducing computational complexity. In the proposed approach, the computational complexity of the system depends only on identifying the k-nearest neighbors for each data sample and running the meta-classifier. Additionally, the study discusses the choice of meta-classifier, recommending the use of Balanced Random Forest.

Keywords: instance selection, data pruning, meta-learning, classification, machine learning

*Corresponding author

Email addresses: marcin.blachnik@polsl.pl (Marcin Blachnik),
piotr.cieplinski@polsl.pl (Piotr Ciepliński)

Preprint submitted to Expert Systems With Applications

December 16, 2024

1. Introduction

Currently, increasing attention is being paid to the issue of filtering data used in the learning process, with a focus on removing useless and noisy data samples. Hence, more and more dedicated solutions are being developed to filter out unimportant samples [1] and constitute the new paradigm called data-centric AI[2]. Additionally, the scaling law turns into the requirement of huge compute power both during training and prediction which is not always applicable in real live scenarios where the compute resources are limited. In that case, both the training data and the prediction model should require small computing resources. Therefore the training set should ensure a possible small size but keep the prediction accuracy of the original training set. This issue is not new, along with its development has been started primarily for the nearest neighbor classifier under the name of instance selection. Thus, already in the late 1960s and early 1970s, algorithms such as Condensed Nearest Neighbor (CNN), Edited Nearest Neighbor (ENN), and many others were developed. The benchmarks of instance selection indicate the Drop3 [3] and ICF [4] algorithms as the most widely used, which, despite not being new, are characterized by excellent properties in terms of the balance between the prediction accuracy of the kNN algorithm and the reduction of the size of the stored set of prototypes (*reduction_rate*) [5]. These algorithms are also applicable not only as elements of the learning process for the kNN algorithm (prototype selection as part of the learning process) and hence could also be used as universal algorithms for reducing the size of the training set for any classifier, thereby accelerating the learning process of complex predictive models, the process of finding optimal parameters, etc. Examples of such applications can be found in [6, 7] or in [5].

All algorithms belonging to the Instance Selection group share one common element, namely, they explore the structure of the nearest neighbors graph by determining connections between the k nearest samples and their interdependencies to identify vectors that are significant for the decision boundary.

This article is also concerned with the challenge of building an instance selection system, however with a different approach by converting the instance

selection process into the binary classification problem. Instance selection algorithms typically rely on the analysis of a nearest-neighbor graph. In the nearest neighbor graph, the vertices correspond to individual instances of the training set, and the edges contain information about the nearest neighbors of a given vector. Instance selection algorithms iteratively, often repeatedly, traverse the graph's vertices, allowing to identify whether a particular vector can be important in the learning process or not. But it is important to note, that the characteristics of the nearest neighbors graph (NNG) are common and do not depend on the domain or problem described by the dataset. Therefore, by extracting features that describe the vertices in the NNG the problem of instance selection can be converted into a new meta-dataset in a fixed-sized feature space (meta-features), that is common to all possible datasets. Now, when taking all previously conducted results of any instance selection algorithm on any given dataset the binary information of keeping or rejecting an instance can be used as instance labels, leading to a meta-training set. This training set can then be used to train a classifier, which will be called a meta-classifier. In prediction mode, a given input dataset for which the instance selection would be performed is represented as an NNG. Then meta-features are extracted and used as input to the meta-classifier which returns a probability indicating the importance of a given instance. This allows us to select instances without running the original instance selection algorithm. To maintain satisfactory generalization of the meta-classifier, it is significant to create a meta-data set that combines meta-data describing the NNG extracted not from one but multiple datasets, and concatenating all these datasets together.

The conducted analyses show that contrasted to traditional methods of instance selection, for some of them, meta-selection methods allow for better results than the original methods. Moreover, these results are achieved much faster, as the selection process is carried out in a single pass, unlike in instance selection methods, where selection requires iteratively traversing the NNG.

An additional advantage of the developed solution is the description of the training samples' importance as a probability density distribution, indicating how likely a given training vector should be included in the training set without affecting the performance of the classifier or in other words how significant a given vector is in the learning process. The use of a thresholding function allows for determining the number of important samples as a post-processing step adjusting the desired value to the needs of a given compute resources.

The contribution of this article:

- A new perspective on the data selection process as a classification problem is presented. In the proposed solution a meta-classifier makes decisions about the importance or non-importance of a given training sample.
- The development of meta-attributes describing the nearest neighbor graph, which also serve as input to the meta-classifier.
- A new metric for instance selection method performance is proposed

Article structure: Section 2 describes the current state of knowledge in the area of data pruning and meta-learning. Section 3 provides a detailed description of the concept, operation, and construction of the meta-classifier, as well as the feature space used to build the meta-classifier. Section 4 presents the experimental design, including the process of building the meta-classifier, performance evaluation methods and metrics used to assess its quality. The next section 5 presents the obtained results in comparison to the original instance selection methods. The final chapter 6 contains a summary of the results and future directions for further research.

2. Related Work

In this article, we focus on and combine two areas of machine learning. On the one hand, the article focusses on the problem of training samples selection, in particular on instance selection problem. On the other hand, we modify the concept of meta-learning to develop a meta-classifier used for data pruning.

In data-centric AI, considerable emphasis is placed on assessing data quality and examining the impact of data on the learning process [8, 9]. A current overview of the state of knowledge in data-centric AI can be found in [10] and [11]. According to the proposed taxonomy, data-centric AI can be divided into training data development, inference data development, and data maintenance. Within training data development, an important area is the issue of dataset size reduction, where the goal is to prepare data so that the model can achieve maximum predictive capabilities by removing noisy instances and eliminating redundant data samples that does not impact the quality of predictive models. This cleaning process should aim to ensure maximum prediction accuracy. For example in [1], the authors discuss scaling laws but note

that better results can be achieved by appropriately cleaning the dataset. As noted in the introduction, the problem of selecting training data is not new, and over the years, many algorithms have been developed, including instance selection methods.

The instance selection process was mainly developed for the nearest neighbor classifier but as shown in [6, 5] it can be used also for other classifiers. Instance selection methods focus on removing noisy samples influencing prediction accuracy, and removing redundant samples so that only the minimal subset of samples assuring high prediction accuracy are stored as the final dataset. For many years since 1960 many methods have been developed utilizing many approaches such as focusing on condensation for example as in CNN [12] or noise removal such as in ENN [13]. Since this early work, many new methods have been developed. An overview of these methods can be found in [14] or in [15]. This comparison points to Drop3 [3], ICF [4], HMN-EI [16], and RMHC algorithms (although the last one is a wrapper method that requires an internal classification model). A more recent summary is available in [17], where the authors compared thirteen different algorithms using text classification as a case study. This research points to algorithms based on local sets such as LSSm[18] and LSBo[18] which were characterized by very high classification accuracy at the cost of size reduction. But also many other algorithms not covered in this comparison were developed. For example in [17] a method based on ranking has been developed where a special score is used to rank samples. Other modern approaches use the so-called global density-based approach for instance selection. In [19] the authors proposed two algorithms based on this approach called global density-based instance selection (GDIS) and its enhanced version called EGDIS. The GDIS algorithm applies a function called relevance function to assess the samples' importance that is based on the purity of k nearest samples and its enhanced version utilized the irrelevance function to improve size reduction. In [20] ensembles of instance selection were proposed where instead of a single method a collection of instance selection methods are evaluated simultaneously and then the obtained results are combined into a single solution. In [21] the concept of boosting was applied to improve the quality of selected instances. A very popular approach to instance selection utilizes meta-heuristics such as in [22] but these methods suffer when the dataset size increases. To overcome this limitation in [23] a solution was proposed such that the dataset is decomposed into subsets using the conditional clustering method and then within each subset an independent instance of genetic algorithm is used, although

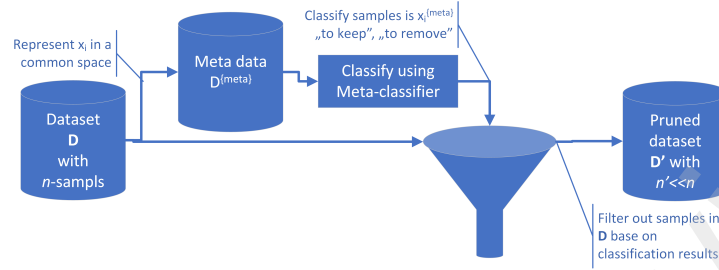


Figure 1: The concept of data processing in the proposed algorithm.

this approach was designed to address the regression problem only.

In parallel to the methods related to data-centric AI, meta-learning methods are being developed. These methods focus on utilizing historical knowledge gained from training models on some other datasets, and use this knowledge to build new predictive models. This enables the creation of models for new datasets faster and with improved predictive accuracy. Additionally, these solutions allow for the identification of models within a set limited time budget to achieve the most efficient model [24, 25]. In meta-model construction, a commonly used concept is leveraging metadata that describes the characteristics of data sets and reference benchmarks models (so-called landmarks - simple models that can be efficiently trained and evaluated) obtained from datasets on which models were previously trained and whose predictive accuracy is already known. Examples of different meta-features can be found, for instance, in [26, 27, 28, 29, 30]. The simplest solutions in meta-learning involve analyzing the similarity of the examined dataset with historical datasets based on meta-features and creating a corresponding ranking of models and its parameters. Another frequently encountered approach is collaborative filtering [31]. Meta-learning methods have also been applied in instance selection [32], although in this case, they have been used to select the most appropriate instance selection method for a given predictive model.

3. Meta-Classifier-Based Instance Selection

The proposed algorithm's basic idea is to transform the problem of instance selection into a classification task, where each sample is classified as "to keep" or "to remove" as presented in Figure 1. To perform such a task, a classifier and a meta-feature space or instance embeddings are needed in which the classifier will operate. At the current stage, the meta-space is

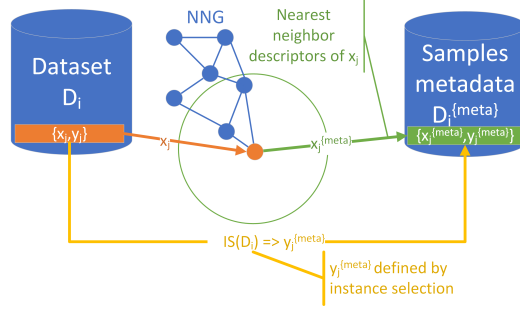


Figure 2: The concept of a single dataset processing in the proposed algorithm. The labelling (marked in yellow) is applied only during the training meta-set preparation.

developed by manually extracting descriptors of the nearest-neighbor graph (NNG), and the meta-classifier is trained on a dataset obtained by combining N independent datasets $\mathbf{D}_i$ each transformed to the meta-space and labeled by the results of the instance selection performed on each of these datasets $\mathbf{D}_i$. The basic idea of meta-space extraction is shown in Figure 2.

Since each of the datasets $\mathbf{D}_i$ is independent each has an independent number of samples n_i so that we can describe it as: $\mathbf{D}_i = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_{n_i}, y_{n_i})\}$, where each pair $(\mathbf{x}, y)$, consists of m_i element input vector (i is specific to a given dataset) $\mathbf{x} \in \mathbb{R}^{m_i}$, and $y \in \{s_1, \dots, l_{c_i}\}$, denotes the label s of the given vector.

Now annotating $\mathbf{G}_i$ as a nearest neighbor graph obtained by $\mathbf{G}_i = NNG(\mathbf{D}_i)$, where the vertices represent individual vectors $\mathbf{x}$, and the edges describe the distances to the nearest neighbors within the dataset $\mathbf{D}_i$. The function $Meta(\mathbf{G})$ extracts features that describe the properties of the vertices in the graph, converting each vertex into a vector $\mathbf{x}^{\{meta\}} \in \mathbb{R}^{n_i^{\{meta\}}}$. By converting each vertex in the NNG, a single meta-dataset is obtained $\mathbf{D}_i^{\{meta\}}$. Hence, it can be written as $\mathbf{D}_i^{\{meta\}} = Meta(NNG(\mathbf{D}_i))$, where both $\mathbf{D}_i$ and $\mathbf{D}_i^{\{meta\}}$ contain the same number of samples. Note that in the meta-set $\mathbf{D}_i^{\{meta\}}$, the number of features $m_i^{\{meta\}}$ is fixed and doesn't depend on i . Finally, by adopting the notation $IS(\cdot)$ as the function performing instance selection, $\mathbf{D}'_i = IS(\mathbf{D}_i)$ the result is a new dataset containing only those samples which are important to the training process, such that $n_i > n'_i$ and often $n_i \gg n'_i$ and the feature space remains unchanged. Then, $\mathbf{D}''_i = \mathbf{D}_i / \mathbf{D}'_i$ contains the irrelevant training samples.

By labeling samples from the meta-set $\mathbf{D}_i^{\{meta\}}$ as

$$\begin{aligned} \forall \left(\mathbf{x}^{\{meta\}} \in \mathbf{D}_i^{\{meta\}} \right) & \text{ if } \mathbf{x}^{\{meta\}} \in \mathbf{D}_i' \\ & \text{ then } y^{\{meta\}} = 1 \\ & \text{ else } y^{\{meta\}} = 0 \end{aligned}$$

we get a binary classification problem. To train the meta-classifier multiple datasets are combined to achieve proper generalization. More formally each of the data sets $\mathbf{D}_i$ undergoes the same transformation procedure transforming it to $\mathbf{D}_i^{\{meta\}}$, and then each of the N meta-data sets $\mathbf{D}_i^{\{meta\}}$ is merged $\mathbf{D}^{\{meta\}} = \bigcup_{i=1}^N \mathbf{D}_i^{\{meta\}}$ into one large meta-data set, on which the meta-model will be trained. The entire procedure of preparing and training the meta-classifier may be written as follows:

1. For each data set $\mathbf{D}_i$, determine the NNG G_i
2. From each G_i determine the meta-set $\mathbf{D}_i^{\{meta\}} = Meta(G_i)$ such that $\mathbf{D}_i \rightarrow \mathbf{D}_i^{\{meta\}}$
3. For each dataset $\mathbf{D}_i$, perform the selection process of vectors $\mathbf{D}_i' = IS(\mathbf{D}_i)$ and determine the labels for the meta-dataset $\mathbf{D}_i^{\{meta\}}$
4. Normalize each of the meta-datasets $Standardize(\mathbf{D}_i^{\{meta\}})$
5. Combine all meta-datasets into one large training meta-dataset
6. Use $\mathbf{D}^{\{meta\}}$ to train meta-classifier

The procedure for applying the meta-classifier is similar and can be described as:

1. For any data set where instance selection $\mathbf{D}_{Te}$ is considered to perform, the dataset should be converted to a meta-set $\mathbf{D}_{Te}^{\{meta\}} = Meta(NNG(\mathbf{D}_{Te}))$
2. Make a prediction using the meta-classifier of samples from $\mathbf{D}_{Te}^{\{meta\}}$, and as prediction results return the probability of belonging to the class "to be removed" marked as $\mathbf{p} = \{p_1, p_2, \dots, p_{n_{Te}}\}$
3. Define the threshold Θ and perform the evaluation assuming the samples labeled as "to be deleted" as the samples for which $p_j < \Theta$, where p_j is the probability that the j 'th vector of the dataset $\mathbf{D}_{Te}$ is to be deleted
4. Remove unnecessary samples from $\mathbf{D}_{Te}$, that is those marked as "to be removed"

Both procedures are presented graphically in Fig. 3. The green color indicates the procedure for creating the meta-training set, the blue color indicates the procedure for training the meta-classifier, and the orange color indicates the procedure for applying the meta instance selection process. Note that the training process is executed ones.

3.1. Meta-Descriptors of the Nearest Neighbor Graph

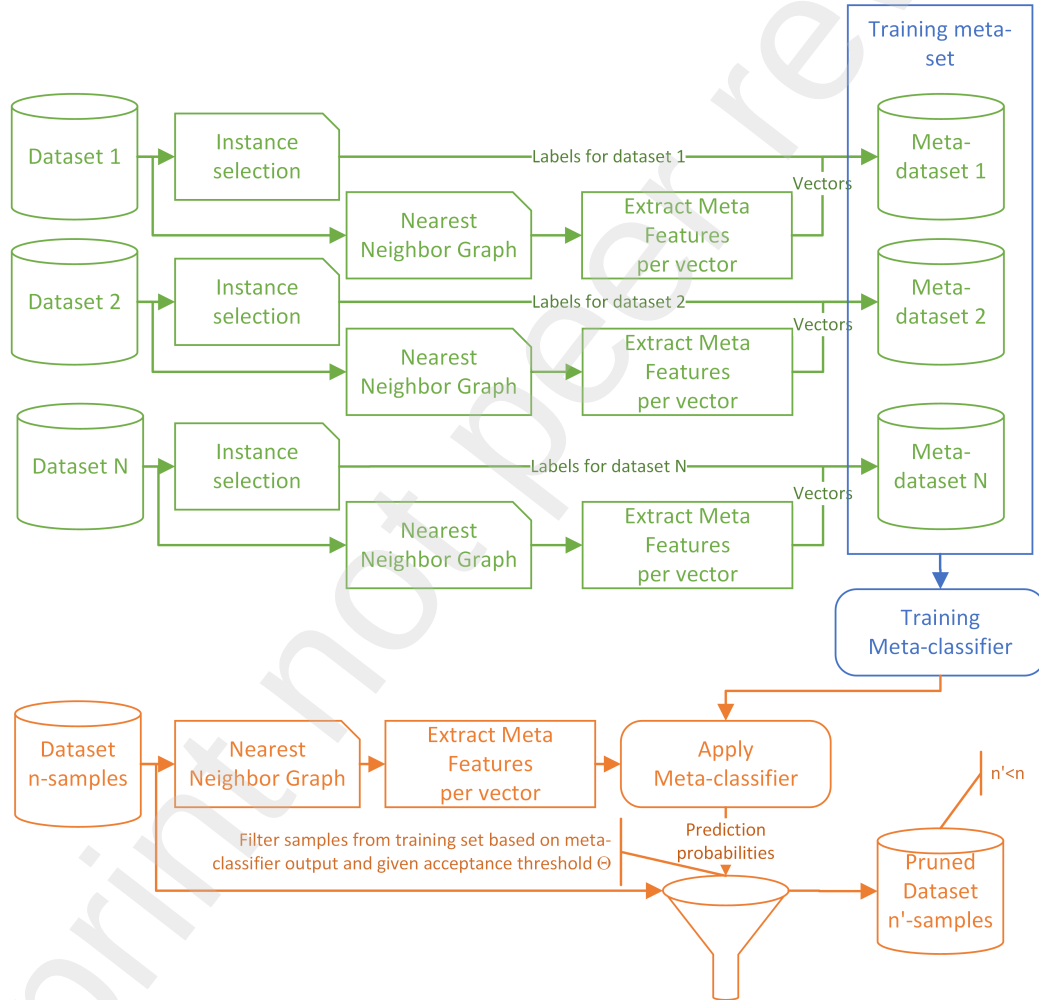
The key elements of the algorithm are meta-descriptors or meta-features that describe the local properties of each node of the nearest neighbor graph. As suggested by Pinto and others in [33] the process of meta-feature extraction can be represented as a triple - an object, a meta-function, and a post-processing. In the proposed solution, the objects are vertexes of the NNG, the function is the length of the edges connected to the given vertex of the NNG, and the post-processing is statistics such as mean, minimum, or count of samples fulfilling certain conditions. The condition is determined by the class label of the query vertex and its neighbors. If the labels agree then the edge is marked as the same class, otherwise, it is marked as the opposite.

- Average distance to k nearest neighbors from the same class
- Average distance to k nearest neighbors from opposite class
- Average distance to k nearest neighbors from any class
- Minimum distance to samples from the same class
- Minimum distance to samples from the opposite class
- Minimum distance to samples from any class
- Number of samples from the same class among k nearest neighbors
- Number of samples from the opposite class among k nearest neighbors

If a given value is missing, for instance there are no objects from the opposite class among k nearest neighbors, the value -1 is inserted. In the calculations a squared euclidean distance is used normalized by the number of features. The normalization is essential to make the distance comparable between datasets of different number of features.

All these features are determined for different values of k . Later in the experiments these values were set to $k = \{3, 5, 9, 15, 23, 33\}$. It is worth

Figure 3: Graphical representation of the meta-classifier training and application. The green color indicates the procedure for creating the meta-training set, the blue color indicates the procedure for training the meta-classifier, and the orange color indicates the procedure for applying the classifier.



noting that from the perspective of computational complexity, it is most effective to conduct the analysis for the maximum value of k (in the given example for $k = 33$), and then extract the results for a smaller value of k by ordering the samples according to the distance (the weight of the edges). This procedure allows for a significant reduction in the computational complexity of the algorithm.

The use of different values of k results from the fact that the proposed solution assumes a single pass over the training samples, thus information about the wider surrounding of a given vector for which meta-features are extracted is necessary. It allows for better identification of significance of given instance. For example, when the reference instance selection algorithm that is used for labeling the meta-set uses $k = 5$ and it is an iterative algorithm which in the next few iterations of samples pruning identifies that a given vector is redundant, because it lies far from the decision boundary, then similar information for the meta-learning system can be approximated using meta-descriptors for wide range of k values. In that scenario, the location of objects from the opposite class can be determined by comparing the results for different values of k . If for $k = 3, 5, 9, 15$ there are no vectors from the opposite class, for $k = 23$ there are few of them, and for $k = 33$ almost half of the nearest samples are objects from the opposite class, the algorithm is able to correctly identify the relative position of this vector and decide to remove it or not.

3.2. Computational Complexity Analysis

Computational complexity can be considered from the perspective of the training process and the application of the developed solution.

The main factor and advantage of the developed solution is the issue of applying the meta instance selection. It consists of two stages. First stage is the procedure of extracting meta-attributes from a new data set $\mathbf{D}_{Te}$ and the second stage is the procedure of applying a meta-classifier. The procedure of extracting meta-attributes depends on the time necessary to create the nearest neighbor graph, which in the worst case requires a computational complexity of $O(n^2)$ (calculating distance between all samples pairs in the dataset $\mathbf{D}_{Te}$ for which instance selection is deliberated to be performed, $|\mathbf{D}_{Te}| = n$). However, using various techniques, e.g. Ball Tree [34] or KDTree [35, 36] algorithms, this complexity can be reduced to $O(n \log(n))$ or using local sensitive hashing [37, 38] methods, it is possible to obtain semi-linear complexity. The selection of the method should depend on the number

of features in the original data set.

The second stage is the prediction time of the meta-classifier. In this paper, it was decided to use algorithms from the Random Forest family, whose prediction time is $K \log(n^*)$, assuming that the forest consists of K trees and the training set of the meta-classifier has n^* training samples $n^* = \sum_i^N n_i$ (note that the computational complexity depends on the size of the training meta-data set, and not on the size of the dataset for which instance selection is deliberated to be performed). In practice, the maximum depth of the tree is often limited to some fixed number for example to $d = 10$, then the complexity is reduced to $K \log(10)$, which can be considered constant from the perspective of the size of the data set $\mathbf{D}_{Te}$. The above indicates that, assuming a constant computational complexity of the classifier, the only computational cost depending on n is determining the nearest neighbors for each of the training samples, which can be limited to log-linear complexity. Importantly, in comparison to classical instance selection algorithms, the selection procedure is no longer an iterative process but consists in a single pass over the samples from $\mathbf{D}_{Te}$ and a single decision on the importance of a given vector.

An independent element of computational complexity is the time required to prepare and train the meta-classifier. Similarly to the application process, that time depends on the process of extracting meta-descriptors of the nearest neighbour graph and on model training. It is worth underlining that the meta-classifier training procedure consists of combining the number of samples of each of the N data sets $\mathbf{D}_i$. As a result, the effective size of the training set $D^{\{meta\}}$ consists of $n^* = n_1 + n_2 + \dots n_N$ training samples. This causes the effective training time to increase radically. Therefore, it was assumed in the studies that the meta-model must be an algorithm that ensures high scalability and low computational complexity of the training process. Such an algorithm is the Random Forest classifier. Additionally it ensures ease of handling imbalanced data by appropriately balancing the size of the sample on which individual trees are trained. As a result, the whole process comes down to complexity $O(N \cdot n^* \log(n^*) + K n^* \log(n^*))$, and the process of meta-descriptor extraction can be easily parallelized by performing the extraction for a single set independently of the other sets in separate processes. Similarly, the Random Forest training process can be easily parallelized, since the individual trees are independent.

4. System and Experimental Design

In order to verify the correctness of the developed algorithm, experiments were conducted comparing the results of several reference instance selection algorithms with the developed solution. The details of the conducted experiments are discussed in the following subsection.

4.1. Reference methods

Since the proposed algorithm requires reference methods that will be used for labeling the samples in the meta-set five different reference instance selection algorithms were used in the experiments. These were namely ENN, Drop3, ICF, HMN-EI, and CCIS. These five methods were selected because these are the most often methods used as a reference for all newly developed algorithms. Below a synthetic description of these methods is provided:

ENN known as Edited Nearest Neighbor algorithm was developed by Wilson and presented in [13]. It directly addresses problem of regularization and noise removal. It starts by assuming that $\mathbf{D}' = \mathbf{D}$, and in subsequent iterations the algorithm runs over vectors from $\mathbf{D}'$, checking whether most of the nearest neighbors belong to the same category as the tested vector. Although it is a fairly simple and old algorithm dating back to 1972, it is commonly used as a component of other solutions, such as Drop3 or ICF as well as effectively applied in imbalanced learning [39].

Drop is a family of algorithms that is a continuation of the concept proposed in the RNN algorithm [40]. The Drop algorithm [41, 3] starts by assuming that initially the selected subset $\mathbf{D}' = \mathbf{D}$, and then each of the instances from $\mathbf{D}'$ is brought to see if its removal will not worsen the prediction accuracy of the kNN classifier. In order to speed up the algorithm, it was proposed to create a nearest neighbours graph storing the $k+1$ nearest neighbors samples including the so called enemies (samples with opposite label) and vectors associated with a given training vector (objects whose given vector is one of k nearest neighbors). Such a structure significantly speeds up the evaluation of individual instances. The family starts with Drop1 algorithm which only evaluates individual instances to see if their removal would degrade the results. Drop2 modifies the Drop1 algorithm by the order of instance removal, so that before the main loop begins, the vectors from $\mathbf{D}'$ are sorted in order

from the largest to the smallest distance to the nearest enemy. This ensures that vectors far from the decision boundary are removed first. Another modification is the Drop3 algorithm, which starts by running the ENN algorithm to remove outlier vectors and boundary samples, these smoothes the decision boundary. Then, the procedure identical to Drop2 begins. The family also contains Drop4 and Drop5, but since they are not used in the experiments the description would be omitted. Apart from, according to many studies, the Drop3 outperforms the other versions.

ICF stands for *Iterative Case Filtering*, is an algorithm similar to the Drop algorithm with the difference that after analyzing individual instances, the algorithm removes all vectors at once [4]. In this algorithm, the authors define the so-called *LocalSet*($\mathbf{x}$) as a set of vectors belonging to a hypersphere whose center is the examined instance $\mathbf{x}$, and the radius is limited by the nearest instance from the opposite class (excluding this instance). On this basis, two sets are defined, named *Coverage*($\mathbf{t}$) and *Reachable*($\mathbf{t}$), respectively, as:

$$\begin{aligned} \text{Coverage}(\mathbf{t}) &= \{\mathbf{x} : \mathbf{x} \in \mathbf{D} \wedge \mathbf{x} \in \text{LocalSet}(\mathbf{t})\} \\ \text{Reachable}(\mathbf{t}) &= \{\mathbf{x} : \mathbf{x} \in \mathbf{D} \wedge \mathbf{t} \in \text{LocalSet}(\mathbf{x})\} \end{aligned} \quad (1)$$

where *Coverage*($\mathbf{t}$) is the set of instances belonging to the set *LocalSet*, whose center is $\mathbf{t}$, or in other words *Coverage*($\mathbf{t}$) is a list of nearest neighbors from the same class that $\mathbf{t}$ belongs to, and *Reachable*($\mathbf{t}$) is the set of instances from $\mathbf{D}$ for which $\mathbf{t}$ belongs to the *LocalSet* of these instances, or equivalently, it is a list of vectors associated with $\mathbf{t}$.

HMN-EI HMN is a family of algorithms, consisting of three solutions called HMN-C, HMN-E and HMN-EI [16]. The whole is based on Hit Miss Network (HMN), which is a modification of the nearest neighbor graph. In HMN, from each vertex there are c edges to the nearest neighbors from each of the c classes (one nearest neighbor to each category). At the same time, HMN has a counter at each node counting the number of hits and misses, where hits represent the associated vectors for which the class label of the nearest neighbor $y_{nn} = y_x$ (y_x the label of vector $\mathbf{x}$), and misses are the scenario in which the nearest neighbor label $y_{nn} \neq y_x$. The individual algorithms HMN-C and HMN-E differ in the rules used to make the decision for removing or leaving instances. In

HMN-E four instead of 1 rule used in HMN-C decide whether to remove or leave an instance, the first of which is responsible for removing, and the others for restoring individual instances. The HMN-EI algorithm, runs the HMN-E procedure until the accuracy of the 1-NN classifier no longer deteriorates.

CCIS CCIS is an algorithm proposed by Elena Marchiori [42] and similarly to algorithms from the HMN family it also uses a hit-miss graph. This algorithm consists of two steps, the first one is called *Class Conditional Selection* (CC) step, while the second one is *Thin-Out Instance Selection* (THIN) step. The CC step is responsible for ensuring proper generalization of the algorithm by removing outliers and boundary vectors. For this purpose, before starting the main loop of the program, candidates to be added to the selected subset $\mathbf{D}'$ are sorted in decreasing order of $Score(a)$ coefficients. Then, the main loop of the CC stage starts, where new candidates are incrementally added to the subset $\mathbf{D}'$ after fulfilling a dedicated condition. The $Score(a)$ function is calculated as the difference:

$$Score(a) = K(p_w, p_b)(a) - K(p_b, p_w)(a) \quad (2)$$

where $K(p_1, p_2)$ is the K-divergence and $p_w(a)$ and $p_b(a)$ are the normalized distributions of the input degree ($inDegree()$) of the within-class and between-class nearest neighbor graph.

All calculations were performed using the implementation of these algorithms created in the Keel project [43] using the Information Selection library [44] available as a plugin for RapidMiner [45]. They are all implemented using Java environment and in the experiments they were set to use $k = 3$ (often used as the default value).

4.2. The Procedure of Assessing the Quality of Instance Selection

The development of tests for the meta-instance selection system required the construction of two sub-processes. One in which the meta-classifier is trained and the other in which it is evaluated on a dataset that has not been used to train the meta-classifier. Having in total N data sets, a leave one out test was applied at the datasets level (leave one dataset out). These tests consisted of excluding one of the N available datasets and using $N-1$ datasets

in the process of preparing the meta-classifier, then an instance selection process was carried out on previously excluded dataset.

During training of the meta-classifier used in meta-instance selection for each of the data sets, firstly the instance selection process was performed using the selected reference algorithm described in the previous section. Secondly, in accordance with the described above procedure, the meta-classifier was trained separately for each of the reference algorithms.

When assessing the quality of instance selection on the single dataset, a 5-fold cross-validation test was performed on that dataset. In this test for each fold, instances were selected using a reference algorithm (ENN, DROP3, HMN-EI, CCIS, ICF), and its performance was assessed using a classifier. Adequately, the meta-instance selection was executed within the same fold, and prediction performance along with the reduction rate was recorded. The performance of instance section methods was assessed using a 1NN classifier (a typical settings for instance selection), that was trained on the selected samples, and evaluated on the test set for each fold.

However, the meta-instance selection algorithm requires defining an acceptance threshold Θ , therefore, within a single fold, the results were collected for different values of $\Theta = \{0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9\}$. In practice, the calculations were repeated each time for each value of Θ .

In addition to the leave-one-dataset-out test presented in the previous above the additional experiments were performed on larger datasets namely CodrnaNorm and covType. Within these experiments, the meta-classifier was trained on meta-samples extracted from all datasets that were used in the previous experiments and evaluated its performance on the new larger datasets of a size of around 500,000 samples. The size was limited because of the execution time of the reference instance selection methods which, for some datasets took a couple of days. Specifically, for Drop3 after 6 days, the algorithm was stopped without reaching the results, thereby the comparison to Drop3 is not shown within the paper.

4.3. Datasets Used in the Experiments

The experiments were performed on data sets available in the Keel project repository [43]. Within this repository, the authors provided data sets of various sizes, from which data sets larger than 2000 samples were selected. Additionally some datasets were obtained from the OpenML project [46]. The full characteristics of the datasets are presented in Table 1. There is need to highlight that these data sets are available in the Keel repository

with a ready division into cross-validation folds. The division provided in the repository the authors used in the experiments.

Table 1: Characteristics of the datasets used in the experiments.

Id	Dataset	# samples	# attr.	# classes
1	banana	5300	2	2
2	electricity	45312	8	2
3	letter	20000	16	26
4	magic	19020	10	2
5	nursery	12960	8	5
6	optdigits	5620	64	10
7	page-blocks	5472	10	5
8	penbased	10992	16	10
9	phoneme	5404	5	2
10	ring	7400	20	2
11	satimage	6435	36	6
12	shuttle	57999	9	7
13	spambase	4597	57	2
14	texture	5500	40	11
15	twonorm	7400	20	2
16	codrnaNorm	488565	8	2
17	covType	581012	54	2

4.4. Performance Metrics

To assess the quality of the final classifier, the classical F1 score and the reduction rate were used. Where the reduction rate is defined as: $reduction_rate = \frac{n-n'}{n}$, where n is the number of samples in the dataset and n' is the number of samples after dataset size reduction.

Since, for the meta-instance selection algorithm it is necessary to define the Θ threshold, instead of a single result returned by the reference algorithm, for meta instance selection a family of results is obtained. In the experiments

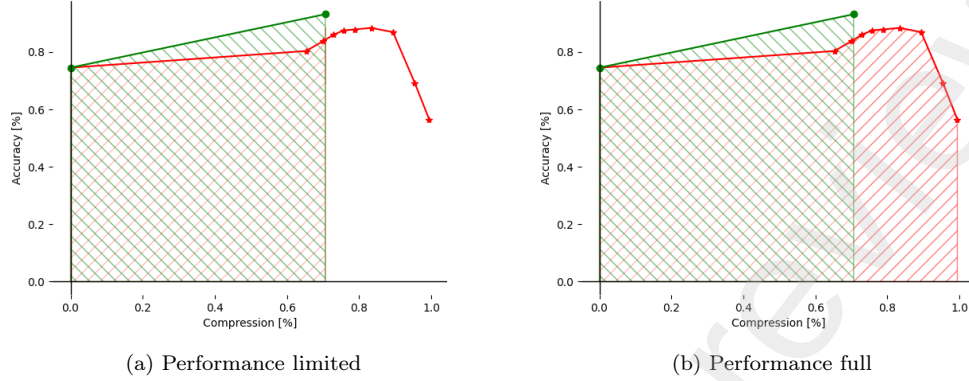


Figure 4: Two types of performance measures used in the experiments. The performance is measured as the area under the accuracy-compression curve. Performance 4a is limited by the reduction rate of the reference instance selection model. Performance 4b is not limited by the reduction rate of the reference instance selection model, it is bounded by the last value of reduction rate. In the figures the area is hatched.

the following calculations were performed for $\Theta = \{0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9\}$. Therefore, for each Θ a pair of (*prediction_performance*, *reduction_rate*) was obtained resulting in a series of results. Examples are shown in Figure 4. This series of results creates a problem when comparing the obtained results with reference instance selection algorithms because the reference methods return a single pair - a single point on the *accuracy* – *reduction_rate* plot. Therefore, in the article, in addition to the typical graphical presentation of the results, they are also evaluated numerically by determining the area under the *accuracy* – *reduction_rate* curve returned by the meta-instance selection algorithm. This area under the *accuracy* – *reduction_rate* also called AUARR is determined by the trapezoidal approximation, starting from the accuracy returned by the model without any instance selection, and then the area is defined by the subsequently collected measurement points obtained for the subsequent values of Θ . This area can be calculated in two ways, one is to limit the maximum *reduction_rate* to the *reduction_rate* obtained by the reference algorithm (see Figure 4a) - denoted as $AUARR_L$. Then both algorithms are compared assuming that the maximum compression is determined by the reference model. However, this comparison does not reflect the full capabilities of the meta-instance selection algorithm, because with its help it is possible to obtain greater compression, therefore the second of the

recorded performance measures is the area under the curve limited only by the last value of the *reduction_rate* coefficient denoted as AUARR. In the case of the reference model, this area is determined by linear approximation between the obtained pair (accuracy, *reduction_rate*) and the accuracy of the 1NN algorithm (Figure 4b).

4.5. Implementation Details

The experiments were conducted using the MetaIS library available on GitHub¹. It implements the developed solution compliant with the API defined by the Scikie-Learn library [47] and the Imabalnced-Learn [48] that extends the basic Scikit-Learn API with the *fit_resample()* method where the instance selection algorithm was implemented. The Random Forest - based methods were considered as a meta-classifier. The selection of Random Forest based algorithm was dictated by considering excellent scalability as a function of the size of the training set. In the proposed solution, the size of the training set increases due to the fact that all partial data sets are glued together. At the same time, due to the high compression of some of the algorithms used in the experiments, the resulting meta-set is unbalanced with the dominance of vectors labeled "to be removed". As a result, the classifiers supporting imbalanced classification was considered as the part of the case study.

5. Results

The experiments were divided into 3 parts. The first part compared the results obtained by the meta instance selection with reference models. Then, the time required for the instance selection obtained by the reference algorithm and meta instance selection was compared and analyzed. Eventually the last part shows the influence of model selection on the meta-classifier and its stability.

5.1. Performance Comparison

Visualization of the performance metrics in particular prediction accuracy and reduction rate are presented in figure 5. In that figures X indicates given reference method and a curve of the same color corresponds to given

¹<https://github.com/mblachnik/MetaIS>

meta equivalent to given method. Therefore, in order to properly analyze these figures the results should be analyzed by color comparing X with a curve of the same color. The results are grouped by dataset. This allows to compare not only the relation between compression and prediction performance but also to identify the best instance selection method for a given dataset. Summarization of these figures by calculating the area under the *accuracy – reduction_rate* curve is shown in Tables 2 and 3.

A comparison of the results shown in the figures indicates two groups of methods, the first which includes ENN and HMN-EI and the second which contains the three remaining methods (Drop3, ICF, and CCIS). The main difference between these methods appears in compression. The first group has significantly lower reduction rate in comparison to the second group. Therefore these two groups will be analyzed separately.

Within the first group, in all of the cases, there were no differences between the reference ENN and HMN-EI methods marked blue and red respectively with their meta equivalents (Meta-ENN and Meta-HMN-EI). Moreover in almost all of the cases, the use of Meta-ENN and meta-HMN-EI allowed us to achieve significantly higher compression almost without any loss in performance, and even with a little gain. Additionally, the shape of the *accuracy – reduction_rate* plot for the meta-instance selection is often flat indicating that even higher compression is possible. The only exception is the Ring data set where Meta-HMN-EI achieved worse accuracy than the reference HMN-EI, but for ENN the accuracy was even higher.

Within the second group, the *accuracy – reduction_rate* curve for the meta-instance selection methods usually indicates a sudden drop after reaching a certain *reduction_rate*. Before that threshold, the curve is usually flat. As a result, after a certain *reduction_rate* the prediction accuracy starts to decrease rapidly. In those scenarios, the reference instance selection methods often allow achieving higher prediction accuracy for its own *reduction_rate*, although for a little less *reduction_rate* the meta-instance selection methods allow to achieve higher prediction accuracy. That phenomenon appears in Letter, Nursery, Optdigits, Page-blocks, Penbased, Ring, Shuttle Texture, and Twonorm datasets.

It is also reflected in the results presented in Table 2, where the area under the *accuracy – reduction_rate* (AUARR) is presented, limited by the *reduction_rate* of the reference instance selection though. The table presents that, the AUARR indicates the average results of the 5-fold cross-validation obtained for each dataset. The last two rows summarize the table. The row

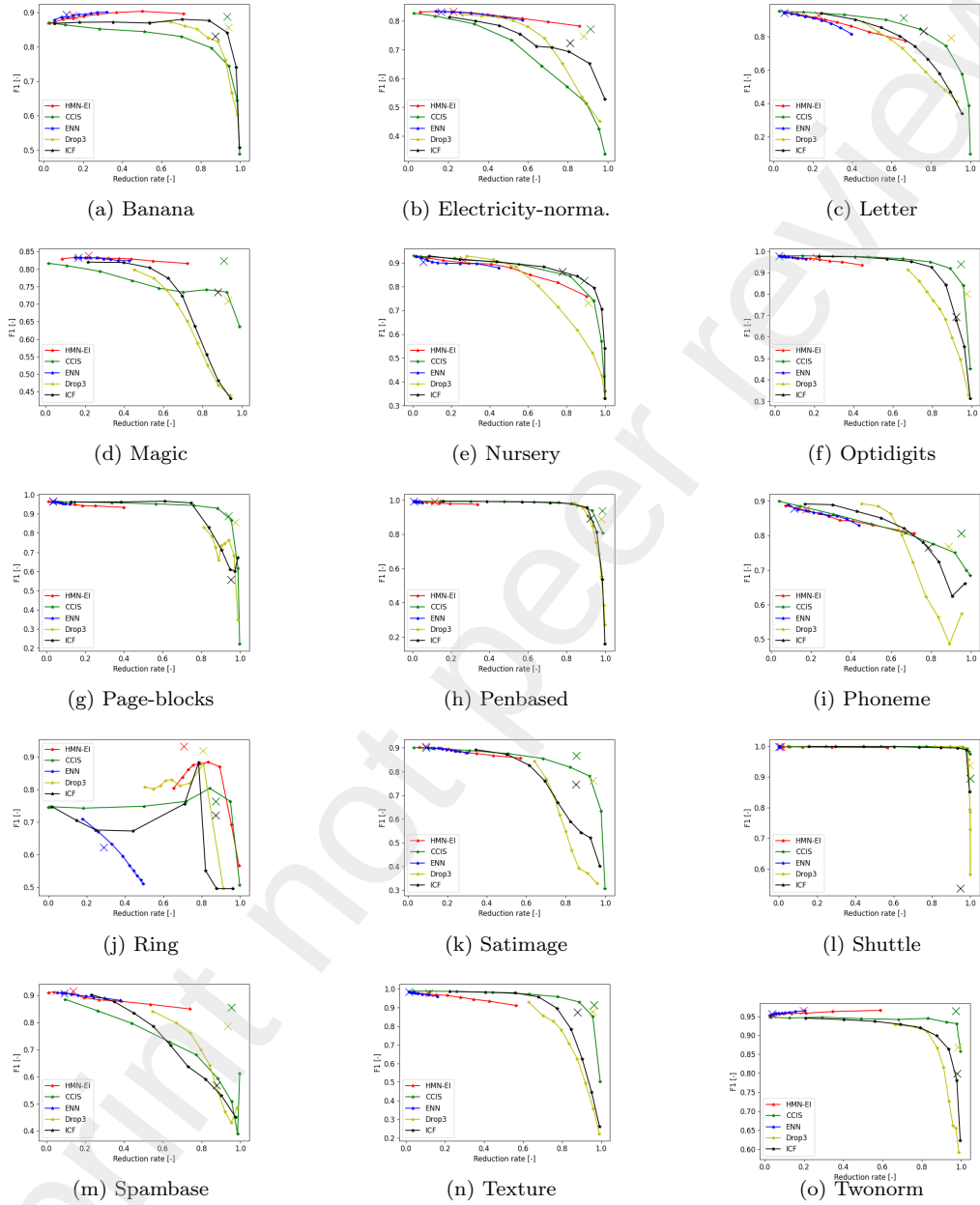


Figure 5: Comparison of the reference instance selection methods (marked as X) with the meta instance selection (marked with a curve) in terms of prediction performance and reduction rate. Colors correspond to a particular reference instance selection method and its corresponding meta-instance selection.

indicated as Wins points to the number of times the MetaIS was significantly better than the reference instance selection method. Here the T-test with $\alpha = 0.05$ was used. The last row represents the average difference between the AUARR obtained for meta-instance selection and the reference instance selection method $AUARR_{MetaIS} - AUARR_{RefIS}$. There shall be noted that the positive values indicate that meta instance selection is better and negative values indicate that the reference instance selection method is better. In the brackets are given the results of the Wilcoxon signed-rank test between meta instance selection and reference instance selection methods (here the results over individual datasets were compared). The (+) indicates that the meta instance selection is statistically significantly better ($\alpha = 0.05$) - here the one-side test was used, and respectively (-) indicates significantly worse results compared to the reference instance selection. The (=) indicates no statistically significant difference. These results indicate that for HMN-EI, ENN, and CCIS there were no statistically significant differences in performance, although, for the HMN-EI method, the MetaHMN-EI won only 3 times, while for the CCIS the MetaCCIS won 9 times and for ENN the MetaENN won 12 times. For the ICF method, the meta-model significantly outperformed the reference ICF method, while for the Drop3 the meta-model was significantly worse.

An extension of these results is presented in Table 3 where the entire AUARR is gathered without limitation by the *reduction_rate* of the reference instance selection. According to these results, the meta-instance selection outperforms the reference instance selection method in almost all of the cases, only for the Drop3 algorithm the difference is not significantly better. It significantly wins with the reference Drop3 8 out of 19 times.

5.2. Execution Time

The execution time is the utmost benefit of the proposed method. To indicate the benefits and speed up of the meta instance selection over the reference methods, the execution time of the experiments discussed in the previous section were recorded. The execution time was measured only on the instance selection process. For the reference instance selection methods, the execution time was recorded using the build-in RapidMiner execution time property of an operator. For the meta instance selection, two different times can be identified that is the training time and selection time. First, the selection time would be analyzed. The obtained results are presented in Table 5, and the speed up is shown in Table 4 and visualized in Figure

Table 2: Comparison of the reference instance selection and the meta instance selection equivalence. The values represented the area under the accuracy - *reduction_rate* curve ($AUARRC_L$) limited by the compression of the reference instance selection. The last two lines summarize the results. The one before the last row indicates the number of times MetaIS was significantly better than the reference model. The last row shows the average difference between the performance of MetaIS and the reference instance selection model over all datasets. The symbols (=), (+), and (-) indicate a statistically significant difference between MetaIS and the reference instance selection. (+) means MetaIS is significantly better than IS, and correspondingly (-) that MetaIS is significantly worse.

Dataset	HMN-EI			ENN			CCIS			ICF			Drop3		
	IS	MetaIS	std	IS	MetaIS	std	IS	MetaIS	std	IS	MetaIS	std	IS	MetaIS	std
banana	0.1023	0.0007	0.1016	0.0008	0.0987	0.0006	0.8210	0.0058	0.7863	0.0001	0.7417	0.0066	0.8006	0.0084	0.8067
electricity-normalized	0.1877	0.0004	0.1880	0.0005	0.1367	0.0004	0.7316	0.0020	0.6533	0.0022	0.6270	0.0021	0.6904	0.0044	0.6767
letter	0.2169	0.0008	0.2135	0.0006	0.0301	0.0001	0.0302	0.0002	0.0180	0.0025	0.0130	0.0019	0.7861	0.0036	0.7253
magic	0.1809	0.0011	0.1812	0.0009	0.1360	0.0007	0.1361	0.0008	0.7460	0.0034	0.6997	0.0037	0.7079	0.0082	0.6756
marketing	0.1438	0.0037	0.1427	0.0036	0.2068	0.0035	0.2042	0.0018	0.0574	0.0015	0.0496	0.0016	0.2365	0.0029	0.2432
nursery	0.2514	0.0041	0.2493	0.0056	0.0626	0.0014	0.0630	0.0015	0.7332	0.0110	0.7483	0.0102	0.7708	0.0234	0.7804
optdigits	0.2029	0.0008	0.2021	0.0006	0.0222	0.0001	0.0222	0.0001	0.9078	0.0051	0.9112	0.0050	0.8650	0.0166	0.8509
page-blocks	0.0317	0.0002	0.0317	0.0002	0.0321	0.0002	0.0321	0.0002	0.8691	0.0086	0.8929	0.0063	0.8849	0.0290	0.8480
penbased	0.1142	0.0002	0.1138	0.0003	0.0070	0.0000	0.0070	0.0000	0.9497	0.0051	0.9658	0.0014	0.9212	0.0075	0.9500
phoneme	0.1521	0.0013	0.1517	0.0014	0.1007	0.0010	0.1010	0.0009	0.8142	0.0048	0.7940	0.0069	0.7458	0.0114	0.7336
ring	0.5909	0.0018	0.5482	0.0045	0.2007	0.0010	0.2083	0.0011	0.6594	0.0024	0.6600	0.0069	0.6722	0.0046	0.6412
satimage	0.0831	0.0003	0.0829	0.0002	0.0859	0.0004	0.0859	0.0005	0.7559	0.0039	0.7459	0.0035	0.7123	0.0073	0.7319
shuttle	0.0111	0.0000	0.0111	0.0000	0.0011	0.0000	0.0011	0.0000	0.9449	0.0158	0.9966	0.0004	0.9652	0.0239	0.9944
spambase	0.1261	0.0007	0.1256	0.0007	0.0862	0.0003	0.0863	0.0004	0.8445	0.0080	0.7326	0.0079	0.7917	0.0195	0.7559
texture	0.1137	0.0005	0.1133	0.0004	0.0139	0.0001	0.0139	0.0001	0.9176	0.0081	0.9340	0.0017	0.8945	0.0099	0.8433
twonorm	0.1354	0.0007	0.1352	0.0007	0.0372	0.0002	0.0371	0.0002	0.9295	0.0033	0.9177	0.0041	0.8607	0.0158	0.9132
codrmaNorm	0.1296	0.0001	0.1289	0.0001	0.0459	0.0000	0.0459	0.0000	0.9303	0.0010	0.9341	0.0006	0.6869	0.0041	0.9177
covtype	0.1521	0.0001	0.1520	0.0001	0.0506	0.0000	0.0507	0.0000	0.8583	0.0008	0.8867	0.0005	0.7052	0.0009	0.7820
Wins	3			12			9			14			5		
Mean(MetaIS-IS)	-0.0028 (-)			0.0003 (=)			-0.0093 (=)			0.0538 (+)			-0.0144 (-)		

Table 3: Comparison of the reference instance selection and the meta instance selection equivalence. The values represented the area under the accuracy - *reduction_rate* curve (*AUARR*). The last two lines summarize the results. The one before the last row indicates the number of times MetaIS was significantly better than the reference model. The last row shows the average difference between the performance of MetaIS and the reference instance selection model over all datasets. The symbols (=), (+), and (-) indicate a statistically significant difference between MetaIS and the reference instance selection. (+) means MetaIS is significantly better than IS, and correspondingly (-) that MetaIS is significantly worse.

Dataset	HMN-EI		CCIS		ENN		ICF		IS		Drop3	
	mean	std	mean	std	mean	std	mean	std	mean	std	mean	std
banana	0.1023	0.0007	0.6350	0.0134	0.0080	0.0009	0.2830	0.0055	0.7417	0.0066	0.8010	0.0076
electricity-normalized	0.1877	0.0004	0.7002	0.0032	0.1364	0.0003	0.4727	0.0030	0.6270	0.0021	0.7401	0.0034
letter	0.2169	0.0008	0.3845	0.0053	0.0501	0.0001	0.3574	0.0014	0.6814	0.0013	0.7773	0.0031
magic	0.1809	0.0011	0.3963	0.0051	0.1350	0.0007	0.3526	0.0034	0.6821	0.0045	0.7009	0.0036
marketing	0.1438	0.0037	0.2914	0.0047	0.1400	0.0022	0.2668	0.0035	0.2771	0.0026	0.2295	0.0023
nursery	0.2514	0.0041	0.7858	0.0212	0.0626	0.0014	0.4021	0.0305	0.7296	0.0103	0.8817	0.0125
optdigits	0.2029	0.0008	0.4241	0.0053	0.0222	0.0001	0.1528	0.0038	0.7081	0.0087	0.9113	0.0036
page-blocks	0.0317	0.0002	0.3769	0.0047	0.0321	0.0002	0.1135	0.0025	0.7205	0.0231	0.9063	0.0127
penbased	0.1142	0.0002	0.3319	0.0070	0.0070	0.0000	0.0505	0.0012	0.8761	0.0015	0.9646	0.0025
phoneme	0.1521	0.0013	0.6084	0.0114	0.1007	0.0010	0.3822	0.0014	0.6592	0.0071	0.8333	0.0084
ring	0.5909	0.0018	0.7857	0.0095	0.2007	0.0004	0.3258	0.0018	0.6426	0.0052	0.6563	0.0100
satimage	0.0831	0.0003	0.5032	0.0041	0.0859	0.0004	0.2694	0.0011	0.7047	0.0075	0.7716	0.0082
shuttle	0.0111	0.0000	0.5646	0.0075	0.0001	0.0000	0.0000	0.0000	0.7261	0.0198	0.9922	0.0012
spambase	0.1261	0.0007	0.6508	0.0113	0.0862	0.0003	0.3416	0.0112	0.6507	0.0165	0.7479	0.0087
texture	0.1137	0.0005	0.5360	0.0072	0.0139	0.0001	0.1571	0.0032	0.8123	0.0017	0.8959	0.0081
twonorm	0.1354	0.0007	0.5655	0.0068	0.0372	0.0002	0.1954	0.0044	0.8007	0.0158	0.9208	0.0039
codrmaNorm	0.1296	0.0001	0.4754	0.0032	0.0459	0.0000	0.1660	0.0009	0.6869	0.0041	0.9278	0.0012
covtype	0.1521	0.0001	0.5230	0.0010	0.0506	0.0000	0.2566	0.0007	0.7052	0.0009	0.8626	0.0006
Wins	17		15		17		17		17		8	
Mean(MetaIS-IS)	0.3896 (+)		0.04499 (+)		0.1771 (+)		0.1173 (+)		0.0042 (=)			

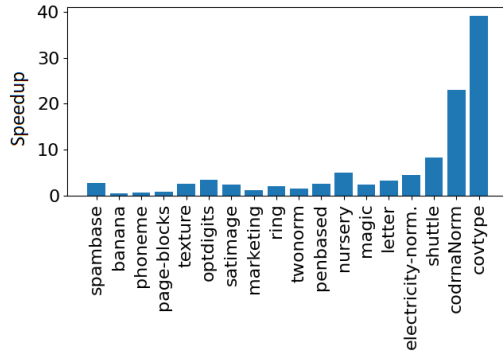
6, although it is important to note that the execution time was measured in two different environments - in the case of reference instance selection the algorithms were implemented in Java using the standard java structures and in terms of meta instance selection the algorithms were implemented in Python using numpy and scipy libraries.

Table 4: Speedup of the meta instance selection algorithm over the reference instance selection method. The values were obtained as a fraction of the reference instance selection time divided by the time needed by its meta-equivalence.

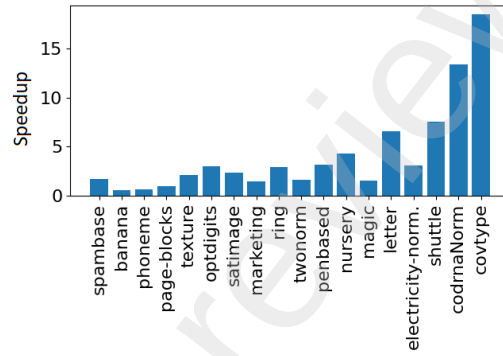
dataset	# samples	HMN-EI [%]	CCIS [%]	ENN [%]	ICF [%]	Drop3 [%]
spambase	4597	1.67	1.35	2.75	2.24	14.09
banana	5300	0.56	0.42	0.50	0.51	7.63
phoneme	5404	0.62	0.59	0.61	0.58	7.08
page-blocks	5472	0.95	0.97	0.91	0.90	14.65
texture	5500	2.11	2.15	2.50	2.30	15.40
optdigits	5620	2.98	2.74	3.53	3.10	20.17
satimage	6435	2.31	2.44	2.48	2.10	16.68
marketing	6876	1.47	1.28	1.25	1.01	1.55
ring	7400	2.90	1.29	2.02	1.42	10.46
twonorm	7400	1.61	1.09	1.52	1.74	7.20
penbased	10992	3.12	2.80	2.66	2.27	21.05
nursery	12960	4.28	3.23	5.00	5.36	36.94
magic	19020	1.57	1.51	2.49	2.37	15.64
letter	20000	6.57	11.69	3.26	3.71	33.37
electricity-norm.	45312	3.08	4.46	4.49	4.88	34.90
shuttle	57999	7.57	4.85	8.29	6.46	214.94
codrnaNorm	488565	13.37	7.06	22.98	11.92	
covtype	581012	18.47	37.97	39.11	69.26	

The analysis of the results presented in Table 4 and visualized in Figure 6 indicates a significant speedup of the meta instance selection algorithm that reaches 219 for the Drop3 algorithm. Moreover, when analyzing Table 5 it can be observed that the execution time of the meta instance selection is independent of the reference algorithm. It is fixed and depends on the meta-feature extraction process and execution of the meta-classifier. This property is essential for the use of the proposed solution together with the very complex and slow instance selection algorithms with high computational complexity.

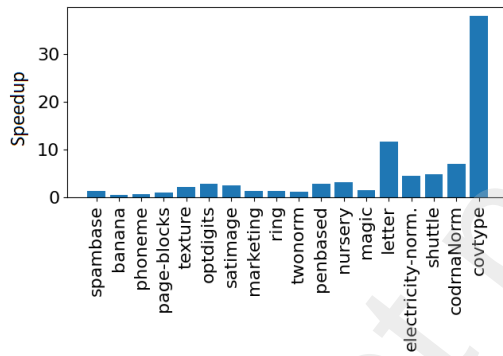
The meta instance selection training time should also be considered since the meta classifier needs to be trained. This process is executed once ergo it does not influence the execution time of the selection phase. The training time may be estimated by the addition of the execution time of the selection phase of all datasets that were presented in Table 5. It may be considered as



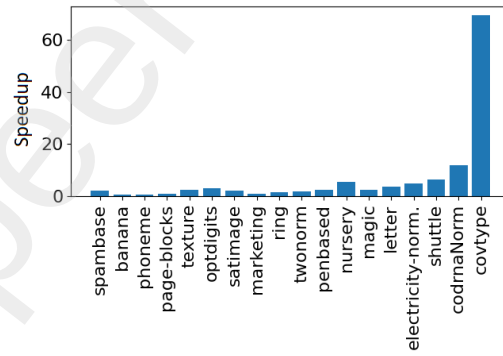
(a) ENN



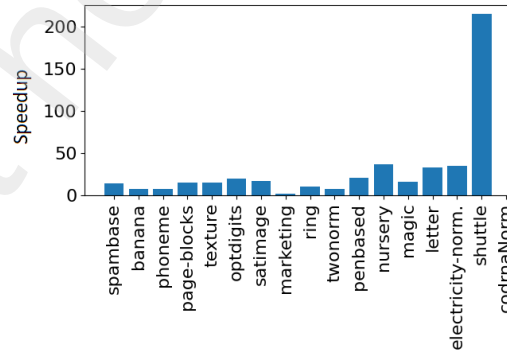
(b) HMN-EI



(c) CCIS



(d) ICF



(e) Drop3

Figure 6: Speedup of the meta instance selection over reference instance selection methods.

Table 5: Comparison of the execution time of the reference instance selection method and the meta instance selection equivalence. The values are represented in seconds. For the codrnaNorm and covtype the values for the Drop3 are missing since after 6 days the reference algorithm didn't converge.

dataset	#samples	HMEI [s]		CCIS [s]		ENN [s]		ICF [s]		Drop3 [s]	
		Ref	Meta	Ref	Meta	Ref	Meta	Ref	Meta	Ref	Meta
spambase	4597	4	3	3	3	7	2	6	3	35	2
banana	5300	1	3	1	3	1	3	1	3	19	2
phoneme	5404	2	3	2	3	2	3	2	3	20	3
page-blocks	5472	3	3	3	3	2	3	3	3	40	3
texture	5500	6	3	6	3	7	3	7	3	42	3
optdigits	5620	8	3	8	3	9	3	9	3	52	3
satimage	6435	8	3	8	3	8	3	7	3	52	3
marketing	6876	5	3	4	3	4	3	4	3	5	3
ring	7400	11	4	5	3	7	3	5	4	36	3
twonorm	7400	6	4	4	3	5	3	6	4	24	3
penbased	10992	15	5	14	5	13	5	11	5	99	5
nursery	12960	29	7	21	7	33	7	36	7	241	7
magic	19020	15	10	14	9	23	9	22	9	147	9
letter	20000	66	10	114	10	30	9	37	10	320	10
electricity-norm.	45312	86	28	120	27	127	28	132	27	950	27
shuttle	57999	268	35	171	35	288	35	228	35	7632	36
codrnaNorm	488565	18393	1375	9396	1330	29637	1290	15469	1298		
covtype	581012	55658	3013	109719	2890	113242	2895	194440	2807		

the reasonable estimation since considering the computational complexity of the random forest classifier and the meta-feature extraction time, this second phase determines the execution time.

5.3. Assessment of the Meta-Classifer Performance

A substantial factor influencing the quality of the meta instance selection is the evaluation and selection of an appropriate meta-classifier. In the conducted studies, models based on the random forest algorithm were adopted as the meta-classifier, mainly due to its scalability as a function of the number of training vectors. However, the problem is the imbalanced distribution of class labels in the meta training set. It appears when the meta training set is labeled by the results of the reference instance selection method with a high value of *reduction_rate*. For example, if the reference instance selection method is characterized by *reduction_rate* = 90%, the label's distribution will also share identical properties. This introduces certain limitations for the constructed classifier. Therefore, a comparison of the standard random forest with the Balanced Random Forest algorithm [39] is made below. A Balanced Random Forest differs from a classical Random Forest in that it

creates a bootstrap sample from the minority class and then samples with replacement an equal number of instances from the majority class, therefore it is also faster during training.

The obtained results for each of the reference instance selection models are presented in Table 6 along with the unbalanced rate calculated as the ratio of the number of samples of the positive class to the total number of samples in the meta training set. For the comparison, three different measures of prediction accuracy were used: average accuracy, average balanced accuracy, and the area under the ROC curve - the so-called AUC coefficient. Since, as mentioned above, the analyzed data set is imbalanced, the meta classifier does not return the binary labels, rather the values of the decision function or class probabilities are returned.

The imbalanced classification problem influences the choice of the model evaluation measure, which has a significant impact on the comparison of results. Therefore, the AUC was used in the study as a measure independent of the acceptance threshold. It allows for a fair comparison of the quality of the prediction performance. The balanced accuracy measure is particularly important since it naturally returns the prediction results taking into account the class label distribution and assures that the model with a higher balanced accuracy rate treats both classes equally. Thus, it naturally offers a proper assessment of the model's accuracy. The ordinary assessment of the prediction accuracy was included to indicate a reference value.

Table 6: Comparison of the performance of Balanced Random Forest and classical Random Forest in application to the design of a meta-classifier.

Model	Classifier	Imbalanced rate [%]	AUC [%]		Balanced Acc [%]		Accuracy [%]	
			Mean	Std	Mean	Std	Mean	Std
HMN-EI	RF	0.8059	0.9872	0.0010	0.9415	0.0020	0.9678	0.0010
	Bal. RF		0.9834	0.0009	0.9351	0.0018	0.9227	0.0013
ENN	RF	0.8919	0.9929	0.0003	0.9288	0.0029	0.9748	0.0009
	Bal. RF		0.9925	0.0003	0.9598	0.0014	0.9490	0.0010
ICF	RF	0.1385	0.9146	0.0024	0.7503	0.0017	0.9155	0.0009
	Bal. RF		0.9119	0.0020	0.8349	0.0031	0.8386	0.0013
CCIS	RF	0.1360	0.9690	0.0011	0.8909	0.0027	0.9659	0.0007
	Bal. RF		0.9581	0.0012	0.8862	0.0025	0.8724	0.0020
Drop3	RF	0.0686	0.9711	0.0018	0.7888	0.0038	0.9626	0.0009
	Bal. RF		0.9695	0.0015	0.9121	0.0030	0.8857	0.0020

The obtained results indicate that in terms of AUC, models based on Balanced Random Forest and classical Random Forests are similar, however, statistical tests always indicated a significant statistical difference (Welch's T-test for $\alpha = 0.05$ was used), indicating the dominance of the Random Forest algorithm. However, the results obtained for balanced accuracy show the opposite results, where, except for the HMN-EI and CCIS algorithms, the Balanced Random Forest algorithm always obtained better results, and the differences reached over 12% in favor of the Balanced Random Forest algorithm (for HMN-EI and CCIS the differences were small at the level of tenths of a percent). The opposite situation occurred in the case of ordinary prediction accuracy, where the differences reached 9% in favor of the Random Forest algorithm. In the conducted research, however, it was decided to use the Balanced Random Forest algorithm, because it provides more balanced results for the minority class, which makes the selection of the acceptance threshold Θ more intuitive, and values at the level of $\Theta = 0.5$ are similar to the results obtained for the reference algorithms. On the other hand, for the classical Random Forest algorithm, the selection of the parameter Θ is a separate issue, which is specific for each basic instance selection algorithm and could be a subject of extended research.

5.4. *Meta-Features Importance Analysis*

The another important element influencing the quality of the obtained results is the selection of the feature space, albeit there is the associated issue as individual meta-features are strongly correlated with each other. This correlation can be visible preeminently for the same function extracting a meta-feature for different values of the k parameter. Hence in the process of analyzing the significance of features, grouping features with similar properties and presenting their sum was performed.

The mean decrease in impurity (MDI) method [49] was used to assess the quality of attributes, which is naturally supported by tree-based algorithms such as Random Forest. The grouping of features was performed based on two aggregation methods. First the meta-features were grouped by the feature types and summed the MDI coefficients for different values of k and the second aggregation method was based on the value of k , then the coefficients for different feature types and the same value of k were summed. The obtained results are presented graphically in Figure 7 and Figure 8, respectively.

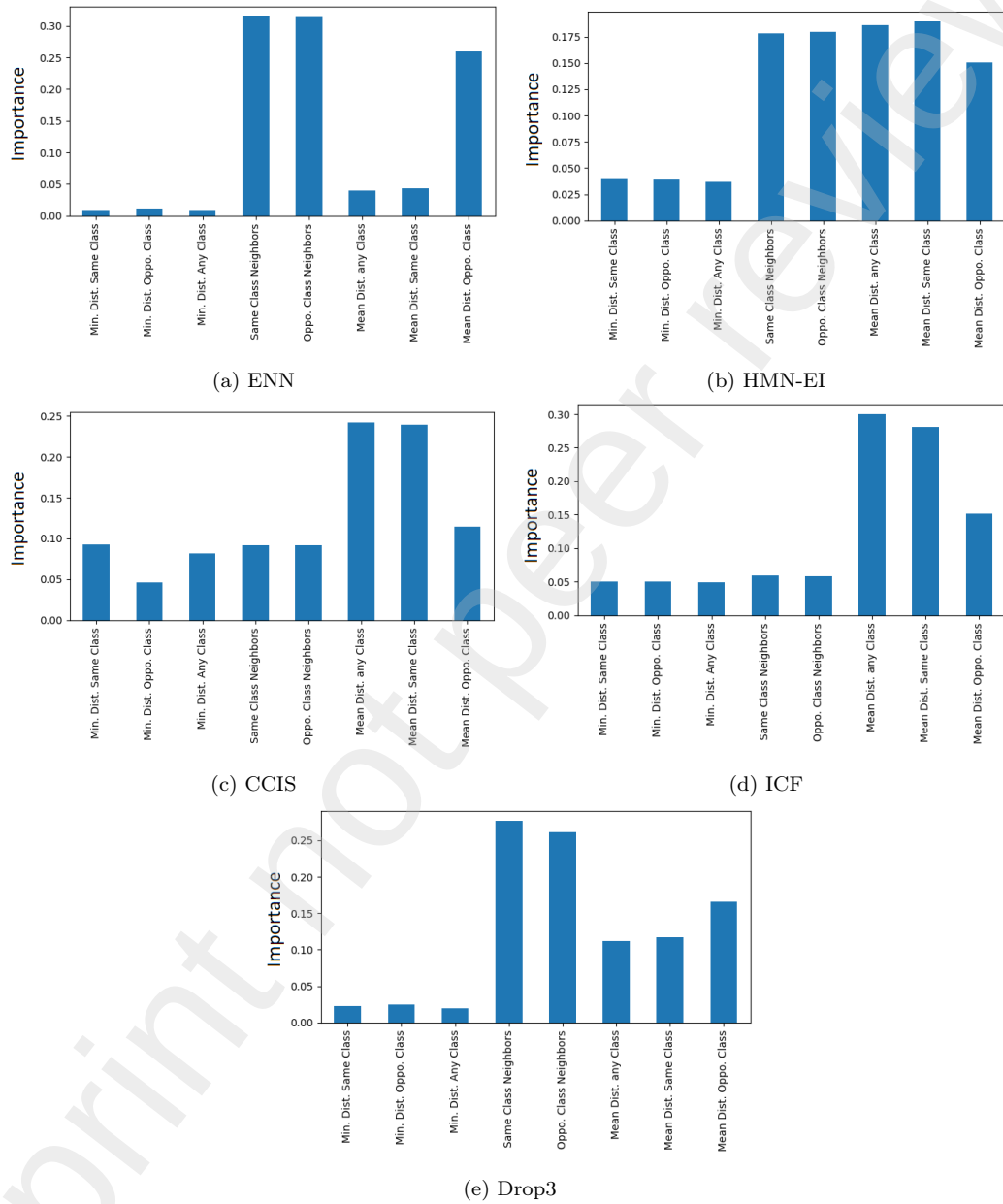


Figure 7: Sum of feature importances obtained after grouping by the feature type.

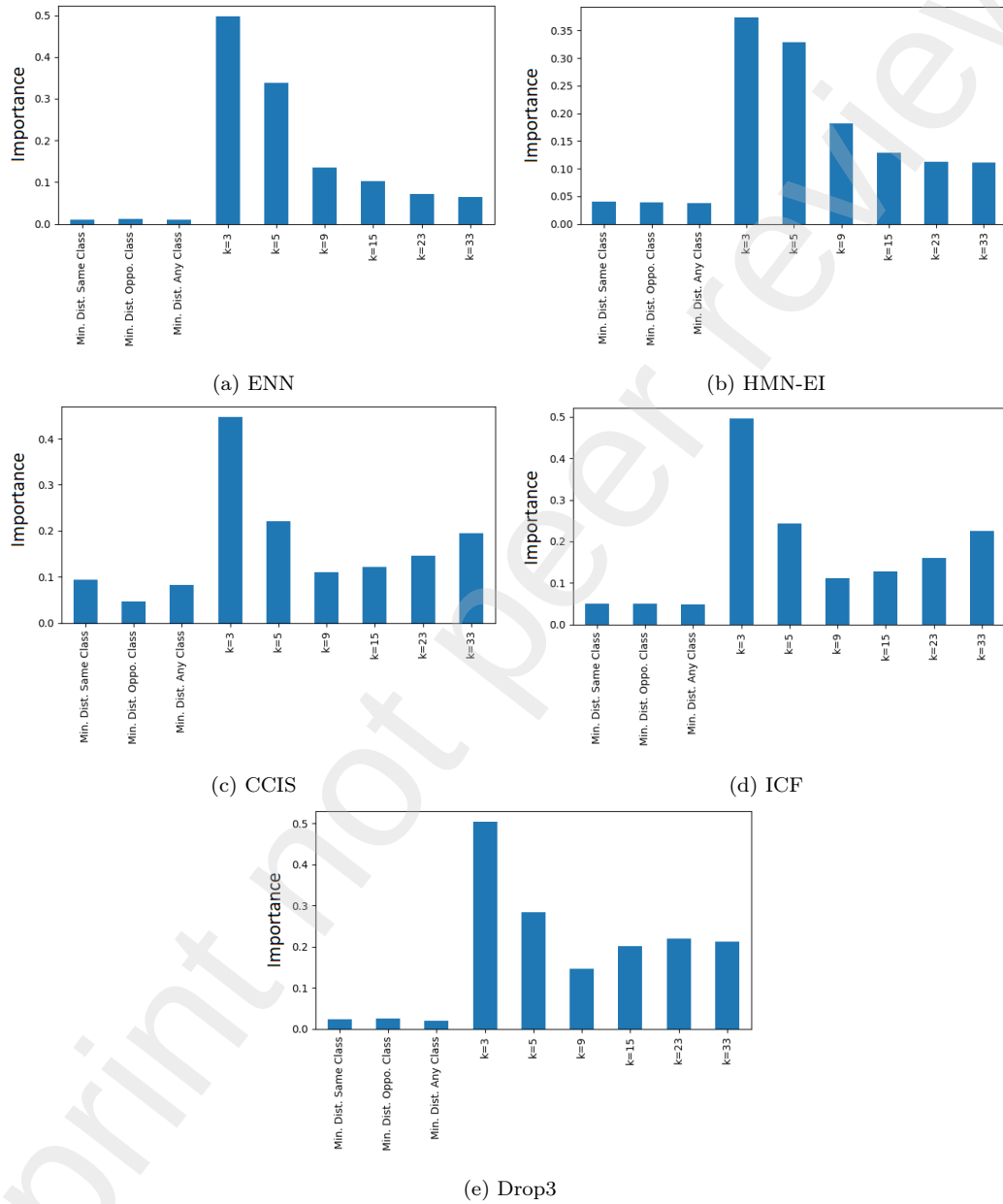


Figure 8: Sum of feature importances obtained after grouping by the feature type.

The obtained results indicate a lack of uniformity of results between the individual meta-models. In terms of attribute grouping by feature types, it can be observed that for the ENN algorithm, the most important features are the number of instances from the same and opposite class among k nearest neighbors. At the same time, the meta-classifier also uses information about the average distance to the opposite class when making decisions. This result is understandable due to the way the ENN algorithm works. In the case of the HMN-EI algorithm, the most important features are a set of meta-attributes describing the statistics of the occurrence of the nearest neighbors belonging to the same and opposite class as well as average distances. Attributes describing the distances to the nearest samples from the opposite class and the same class are ignored here. The above corresponds to the statistics analyzed by the Hit Miss Network, which is the basis of the HME-EI algorithm. For the CCIS algorithm, the most important attribute is also the average distance to the nearest neighbors, including those from the same class, but the remaining attributes of the dataset are also relatively important especially when compared to the other methods. The attribute importances for the ICF algorithm present a very similar characteristic to CCIS. In this case, a criterion such as Coverage can be relatively easily identified, but the Reachable criterion must be estimated based on the remaining features. In the case of the Drop3 algorithm, the attribute importances are similar to the ENN algorithm, which is justified, because the Drop3 algorithm starts with the ENN algorithm, followed by the proper instance selection procedure. At the same time, due to the need to arrange the removed vectors in order defined by the distance to the nearest enemy, the following most important attribute is the average distance to the nearest instance from the opposite class.

In the case of grouping attribute importance by parameter k , it can be observed that usually the most important feature is $k = 3$. This is apprehensible since each of these algorithms was initialized specifically for $k = 3$. For the ENN and HMN-EI algorithms, the decrease in attribute importance for increasing values of k may be observed. On the other hand, for the CCIS, ICF and Drop3 algorithms, attribute importance for subsequent values of k reaches a minimum for $k = 9$, after which there is a gradual increase in attribute importance for higher k . The above results from the iterative nature of the algorithms. These algorithms complete the selection process in subsequent iterations, in this case increasing the number of k allows for a better estimation of the selection process results.

6. Conclusions

This paper presents the results of research related to the training set selection system using meta-features and a meta-classifier. The developed method consists of converting classical instance selection methods into the binary classification problem where each training sample is classified as "to be removed"/"to be kept" using any classifier operating in the meta-feature space. These meta-features represent samples of any data set in a common feature space that is extracted from the nearest neighbors graph. As shown in the paper, this methodology allows for significant acceleration of the calculations, due to the fact that results may be obtained with a single-pass analysis of each of the vectors, without the need to implement an iterative process. It was also shown that the proposed method is generic and can be used with any instance selection method utilizing a nearest-neighbor graph for instance assessment. Furthermore, the developed solution allows for dynamic adjustment of the size of the resulting training set, because the value returned from the system represents the classifier's decision function or the value of the probability of belonging to the "to be removed" class. Then, the user can achieve the desired training set compression by setting appropriate acceptance threshold Θ .

When building a meta-classifier and dealing with imbalanced classification the Balanced Random Forest classifier is a better choice than the classical Random Forest, because it facilitates the selection of the appropriate value of the Θ parameter.

Finally, the results of meta-features importance show that different methods have particular preferences and focus on various characteristics of the NNG. This opens a room for feature research. For example, one of the options is the ensemble model. Since the the meta-classifier for particular instance selection methods focuses on different feature subsets the obtained ensemble should have a guaranteed diversity. Due to the breadth of this problem, the above will go beyond the scope of this study thus it is not presented within the scope of the article.

7. Acknowledgements

Publication supported by the Excellence Initiative – Research University program implemented at the Silesian University of Technology, year 2024

References

- [1] B. Sorscher, R. Geirhos, S. Shekhar, S. Ganguli, A. Morcos, Beyond neural scaling laws: beating power law scaling via data pruning, *Advances in Neural Information Processing Systems* 35 (2022) 19523–19536.
- [2] M. H. Jarrahi, A. Memariani, S. Guha, The principles of data-centric ai, *Communications of the ACM* 66 (8) (2023) 84–92.
- [3] D. Wilson, T. Martinez, Reduction techniques for instance-based learning algorithms, *ML* 38 (2000) 257–268.
- [4] H. Brighton, C. Mellish, Advances in instance selection for instance-based learning algorithms, *Data mining and knowledge discovery* 6 (2) (2002) 153–172.
- [5] S. García, J. Luengo, F. Herrera, Tutorial on practical tips of the most influential data preprocessing algorithms in data mining, *Knowledge-Based Systems* 98 (2016) 1–29.
- [6] M. Blachnik, M. Kordos, Comparison of instance selection and construction methods with various classifiers, *Applied Sciences* 10 (11) (2020) 3933.
- [7] M. Kordos, A. Rusiecki, Reducing noise impact on mlp training, *Soft Computing* 20 (1) (2016) 49–65.
- [8] B. Saha, D. Srivastava, Data quality: The other face of big data, in: 2014 IEEE 30th international conference on data engineering, IEEE, 2014, pp. 1294–1297.
- [9] A. Jain, H. Patel, L. Nagalapatti, N. Gupta, S. Mehta, S. Guttula, S. Mujumdar, S. Afzal, R. Sharma Mittal, V. Munigala, Overview and importance of data quality for machine learning tasks, in: *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, 2020, pp. 3561–3562.
- [10] P. Singh, Systematic review of data-centric approaches in artificial intelligence and machine learning, *Data Science and Management* 6 (3) (2023) 144–157.

- [11] D. Zha, Z. P. Bhat, K.-H. Lai, F. Yang, Z. Jiang, S. Zhong, X. Hu, Data-centric artificial intelligence: A survey, arXiv preprint arXiv:2303.10158 (2023).
- [12] P. Hart, The condensed nearest neighbor rule., IEEE Trans. on Information Theory 16 (1968) 515–516.
- [13] D. Wilson, Asymptotic properties of nearest neighbour rules using edited data., IEEE Trans. on Systems, Man, and Cybernetics SMC-2 (1972) 408–421.
- [14] N. Jankowski, M. Grochowski, Comparison of instance selection algorithms. i. algorithms survey, LNCS 3070 (2004) 598–603.
- [15] S. García, J. Derrac, J. R. Cano, F. Herrera, Prototype selection for nearest neighbor classification: Taxonomy and empirical study, Pattern Analysis and Machine Intelligence, IEEE Transactions on 34 (3) (2012) 417–435.
- [16] E. Marchiori, Hit miss networks with applications to instance selection, Journal of Machine Learning Research 9 (Jun) (2008) 997–1017.
- [17] W. Cunha, F. Viegas, C. França, T. Rosa, L. Rocha, M. A. Gonçalves, A comparative survey of instance selection methods applied to non-neural and transformer-based text classification, ACM Computing Surveys 55 (13s) (2023) 1–52.
- [18] E. Leyva, A. González, R. Pérez, Three new instance selection methods based on local sets: A comparative study with several approaches from a bi-objective perspective, Pattern Recognition 48 (4) (2015) 1523–1537.
- [19] M. Malhat, M. El Menshawy, H. Mousa, A. El Sisi, A new approach for instance selection: Algorithms, evaluation, and comparisons, Expert Systems with Applications 149 (2020) 113297.
- [20] M. Blachnik, Ensembles of instance selection methods. a comparative study., International Journal of Applied Mathematics and Computer Science 29 (1) (2019).
- [21] A. de Haro-García, G. Cerruela-García, N. García-Pedrajas, Instance selection based on boosting for instance-based learners, Pattern Recognition 96 (2019) 106959.

- [22] S. M. H. Mousavi, V. Charles, T. Gherman, An evolutionary pentagon support vector finder method, *Expert Systems with Applications* 150 (2020) 113284.
- [23] M. Kordos, M. Blachnik, R. Scherer, Fuzzy clustering decomposition of genetic algorithm-based instance selection for regression problems, *Information Sciences* 587 (2022) 23–40.
- [24] J. N. van Rijn, S. M. Abdulrahman, P. Brazdil, J. Vanschoren, Fast algorithm selection using learning curves, in: *Advances in Intelligent Data Analysis XIV: 14th International Symposium, IDA 2015, Saint Etienne. France, October 22-24, 2015. Proceedings 14*, Springer, 2015, pp. 298–309.
- [25] S. M. Abdulrahman, P. Brazdil, J. N. van Rijn, J. Vanschoren, Speeding up algorithm selection using average ranking and active testing by introducing runtime, *Machine learning* 107 (2018) 79–108.
- [26] Y. Peng, P. A. Flach, C. Soares, P. Brazdil, Improved dataset characterisation for meta-learning, in: *International Conference on Discovery Science*, Springer, 2002, pp. 141–152.
- [27] C. Castiello, G. Castellano, A. Fanelli, Meta-data: Characterization of input features for meta-learning, in: *Proceedings of the Second International Conference on Modeling Decisions for Artificial Intelligence, LNAI 3558*, 2005, pp. 457–468.
- [28] M. Reif, F. Shafait, A. Dengel, Meta-learning for evolutionary parameter optimization of classifiers, *Machine Learning* 87 (3) (2012) 357–380.
- [29] R. Leite, P. Brazdil, Exploiting performance-based similarity between datasets in metalearning, in: *AAAI Workshop on Meta-Learning and MetaDL Challenge*, PMLR, 2021, pp. 90–99.
- [30] A. Rivolli, L. P. Garcia, C. Soares, J. Vanschoren, A. C. de Carvalho, Meta-features for meta-learning, *Knowledge-Based Systems* 240 (2022) 108101.
- [31] T. Wei, Z. Wu, R. Li, Z. Hu, F. Feng, X. He, Y. Sun, W. Wang, Fast adaptation for cold-start collaborative filtering with meta-learning, in:

- 2020 IEEE International Conference on Data Mining (ICDM), IEEE, 2020, pp. 661–670.
- [32] E. Leyva, Y. Caisés, A. González, R. Pérez, On the use of meta-learning for instance selection: An architecture and an experimental study, *Information Sciences* 266 (2014) 16–30.
 - [33] F. Pinto, C. Soares, J. Mendes-Moreira, Towards automatic generation of metafeatures, in: *Pacific-Asia Conference on Knowledge Discovery and Data Mining*, Springer, 2016, pp. 215–226.
 - [34] O. Chubet, P. Parikh, D. R. Sheehy, S. Sheth, Proximity search in the greedy tree, in: *Symposium on Simplicity in Algorithms (SOSA)*, SIAM, 2023, pp. 332–342.
 - [35] P. Ram, K. Sinha, Revisiting kd-tree for nearest neighbor search, in: *Proceedings of the 25th acm sigkdd international conference on knowledge discovery & data mining*, 2019, pp. 1378–1388.
 - [36] Y. Chen, L. Zhou, Y. Tang, J. P. Singh, N. Bouguila, C. Wang, H. Wang, J. Du, Fast neighbor search by using revised kd tree, *Information Sciences* 472 (2019) 145–162.
 - [37] D. Cheng, J. Huang, S. Zhang, Q. Wu, A robust method based on locality sensitive hashing for k-nearest neighbors searching, *Wireless Networks* 30 (5) (2024) 4195–4208.
 - [38] O. Jafari, P. Maurya, P. Nagarkar, K. M. Islam, C. Crushev, A survey on locality sensitive hashing algorithms and their applications, *arXiv preprint arXiv:2102.08942* (2021).
 - [39] G. Lemaître, F. Nogueira, C. K. Aridas, Imbalanced-learn: A python toolbox to tackle the curse of imbalanced datasets in machine learning, *Journal of Machine Learning Research* 18 (17) (2017) 1–5.
URL <http://jmlr.org/papers/v18/16-365.html>
 - [40] G. Gates, The reduced nearest neighbor rule (corresp.), *IEEE transactions on information theory* 18 (3) (1972) 431–433.
 - [41] D. R. Wilson, T. R. Martinez, Instance pruning techniques, in: *ICML*, Vol. 97, 1997, pp. 403–411.

- [42] E. Marchiori, Class conditional nearest neighbor for large margin instance selection, *IEEE Transactions on Pattern Analysis and Machine Intelligence* 32 (2) (2010) 364–370.
- [43] I. Triguero, S. González, J. M. Moyano, S. García, J. Alcalá-Fdez, J. Lugo, A. Fernández, M. J. del Jesús, L. Sánchez, F. Herrera, Keel 3.0: an open source software for multi-stage analysis in data mining, *International Journal of Computational Intelligence Systems* 10 (1) (2017) 1238–1249.
- [44] M. Blachnik, M. Kordos, Information selection and data compression rapidminer library, in: *Machine Intelligence and Big Data in Industry*, Springer, 2016, pp. 135–145.
- [45] M. Hofmann, R. Klinkenberg, *RapidMiner: Data mining use cases and business analytics applications*, CRC Press, 2016.
- [46] M. Feurer, J. N. Van Rijn, A. Kadra, P. Gijsbers, N. Mallik, S. Ravi, A. Müller, J. Vanschoren, F. Hutter, Openml-python: an extensible python api for openml, *Journal of Machine Learning Research* 22 (100) (2021) 1–5.
- [47] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, E. Duchesnay, Scikit-learn: Machine learning in Python, *Journal of Machine Learning Research* 12 (2011) 2825–2830.
- [48] G. Lemaître, F. Nogueira, C. K. Aridas, Imbalanced-learn: A python toolbox to tackle the curse of imbalanced datasets in machine learning, *Journal of Machine Learning Research* 18 (17) (2017) 1–5.
URL <http://jmlr.org/papers/v18/16-365.html>
- [49] G. Louppe, L. Wehenkel, A. Sutera, P. Geurts, Understanding variable importances in forests of randomized trees, *Advances in neural information processing systems* 26 (2013).